

AEM Sling Servlets

Complete Guide — Types, Registration & Differences

This guide covers everything discussed in the AEM Sling Servlets session — types of servlets, how to register them, and a clear explanation of the differences between Resource Type and Path-Based servlets with real examples.

1. What is a Sling Servlet?

When a request hits AEM, **Sling resolves which servlet to call** based on the URL structure. Sling breaks the URL into parts and finds the best matching servlet.

URL Decomposition

URL Example	Resource	Selectors	Extension
/content/page.json	/content/page	none	json
/content/page.data.json	/content/page	data	json
/content/page.feed.data.xml	/content/page	feed, data	xml

2. Types of Sling Servlets in AEM

2a. Resource Type Servlet (Recommended)

Fires when a resource with a specific **sling:resourceType** is requested with a matching selector and extension. It is the preferred, more secure approach.

```
@Component(service = Servlet.class)
@SlingServletResourceTypes(
    resourceTypes = "myapp/components/product",
    methods       = HttpConstants.METHOD_GET,
    selectors     = "data",
    extensions    = "json"
)
public class ProductDataServlet extends SlingSafeMethodsServlet {
    @Override
    protected void doGet(SlingHttpServletRequest req,
                        SlingHttpServletResponse res) throws IOException {
        Resource resource = req.getResource();
        ValueMap props    = resource.getValueMap();
        String productName = props.get("productName", "Unknown");
        String price       = props.get("price", "0.00");

        res.setContentType("application/json");
        res.getWriter().write(
            "{\"productName\": \"" + productName + "\", "
            + "\"price\": \"" + price + "\"}"
        );
    }
}
```

Called via: /content/mysite/products/shirt/jcr:content.data.json

2b. Path-Based Servlet

Bound to a **fixed URL path** like /bin/myapp/search. Not tied to any content resource. Anyone can call this URL directly.

```
@Component(service = Servlet.class)
@SlingServletPaths("/bin/myapp/search")
public class SearchServlet extends SlingAllMethodsServlet {
    @Override
    protected void doGet(SlingHttpServletRequest req,
                        SlingHttpServletResponse res) throws IOException {
        String keyword = req.getParameter("keyword");
        res.setContentType("application/json");
        res.getWriter().write("{\"keyword\": \"" + keyword + "\"}");
    }
}
```

Called via: /bin/myapp/search?keyword=shirt

2c. Choosing the Right Base Class

Base Class	HTTP Methods	When to Use
SlingSafeMethodsServlet	GET, HEAD only	Read-only data fetching
SlingAllMethodsServlet	GET, POST, PUT, DELETE, etc.	Write / form operations

3. How to Register Sling Servlets

Method 1 — @SlingServletResourceTypes (Modern, Preferred)

```
@Component(service = Servlet.class) // Register as OSGi Servlet service
@slingServletResourceTypes(
    resourceTypes = "myapp/components/mycomponent",
    methods       = HttpConstants.METHOD_GET,
    extensions    = "json",
    selectors     = "data"
)
public class MyServlet extends SlingSafeMethodsServlet { ... }
```

Method 2 — @SlingServletPaths

```
@Component(service = Servlet.class)
@slingServletPaths("/bin/myapp/myservlet")
public class MyPathServlet extends SlingAllMethodsServlet { ... }
```

Method 3 — Legacy OSGi Properties (Older AEM)

```
@Component(
    service = Servlet.class,
    property = {
        "sling.servlet.resourceTypes=myapp/components/mycomponent",
        "sling.servlet.methods=GET",
        "sling.servlet.extensions=json",
        "sling.servlet.selectors=data"
    }
)
public class LegacyServlet extends SlingSafeMethodsServlet { ... }
```

Registration Properties Reference

OSGi Property	Annotation Attribute	Description	Example
sling.servlet.resourceTypes	resourceTypes	Resource type to bind	myapp/components/page
sling.servlet.paths	@SlingServletPaths	Absolute URL path	/bin/myservlet
sling.servlet.methods	methods	HTTP method	GET, POST
sling.servlet.extensions	extensions	URL extension	json, html
sling.servlet.selectors	selectors	URL selector	data, feed

Required Maven Dependencies

```
<dependency>
  <groupId>org.apache.sling</groupId>
  <artifactId>org.apache.sling.api</artifactId>
  <version>2.27.2</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.sling</groupId>
  <artifactId>org.apache.sling.servlets.annotations</artifactId>
  <version>1.2.6</version>
  <scope>provided</scope>
</dependency>
```

4. Resource Path vs Resource Type — The Core Difference

This is the most important concept. Before understanding which servlet fires, you must know the difference between these two things.

Resource Path

The **location/address** of a node in the JCR repository. Think of it like a file path on your computer — it just tells you **WHERE** the content lives.

```
/content/mysite/products/blue-shirt ← Resource Path
/content/mysite/products/red-shoes ← Resource Path
/content/dam/mysite/images/logo.png ← Resource Path
```

Resource Type

Every node has a **sling:resourceType** property. This tells AEM **WHAT kind of thing** this content is — like a label or category.

```
/content/mysite/products/blue-shirt
  jcr:content
    sling:resourceType = "myapp/components/product" ← Resource Type
    productName       = "Blue Shirt"
    price              = "29.99"

/content/mysite/articles/my-blog
  jcr:content
    sling:resourceType = "myapp/components/article" ← Resource Type
    title               = "My Blog Post"
```

Resource Path = WHERE the content is (/content/mysite/products/blue-shirt) Resource Type = WHAT the content is (myapp/components/product)

5. Resource Type vs Path-Based — The Analogy

The Restaurant Analogy

Path-Based Servlet = A General Help Desk

There is one fixed counter at a known location. Anyone who walks in and goes to Counter 5 gets served — it doesn't matter who they are or where they came from. The counter is always there at that fixed spot.

Resource Type Servlet = A Specialist Called to Your Table

No fixed counter exists. The waiter looks at what type of customer you are and calls the right specialist to your table. VIP Guest gets the VIP Specialist. Vegan Guest gets the Vegan Chef.

In AEM Terms

```
-- PATH-BASED --
Anyone hitting /bin/myapp/search → SearchServlet responds
```

(doesn't care about content type at all)

-- RESOURCE TYPE --

Page with resourceType=myapp/components/product → ProductServlet

Page with resourceType=myapp/components/article → ArticleServlet

Page with resourceType=myapp/components/product → ProductServlet again

One Final Mental Model

Path-Based: Like a phone number everyone knows. Anyone can dial /bin/myapp/search and get through. The servlet doesn't know or care who you are.

Resource Type: Like a doctor assigned to your patient record. The doctor only shows up when YOUR specific record (resourceType) is opened. A different patient gets a different doctor.

6. Full Comparison

	Resource Type Servlet	Path-Based Servlet
Binding	slings:resourceType of a node	Fixed URL path /bin/...
URL example	/content/page.data.json	/bin/myapp/api
Best for	Component-specific data	Standalone APIs, form handlers
Tied to content?	YES — reads the JCR node directly	NO — works independently
Security	Safer by default	Must whitelist in Dispatcher
Selector support	YES	NO
Recommended	YES (preferred approach)	Only when needed

7. Real Example — POST Servlet (Contact Form)

A Contact Us form that submits user data and saves it to the JCR.

```
@Component(service = Servlet.class)
@SlingServletPaths("/bin/myapp/contact")
public class ContactFormServlet extends SlingAllMethodsServlet {

    @Override
    protected void doPost(SlingHttpServletRequest req,
                          SlingHttpServletResponse res)
        throws ServletException, IOException {

        // Step 1: Read form fields
        String name    = req.getParameter("name");
        String email   = req.getParameter("email");
        String message = req.getParameter("message");
```

```

// Step 2: Validate
if (name == null || email == null || message == null) {
    res.setStatus(400);
    res.getWriter().write("{\"error\": \"All fields required\"}");
    return;
}

// Step 3: Save to JCR
ResourceResolver resolver = req.getResourceResolver();
Resource folder = resolver.getResource("/content/mysite/submissions");
Map<String, Object> props = new HashMap<>();
props.put("jcr:primaryType", "nt:unstructured");
props.put("name", name);
props.put("email", email);
props.put("message", message);
String nodeName = "submission-" + System.currentTimeMillis();
resolver.create(folder, nodeName, props);
resolver.commit();

// Step 4: Respond
res.setContentType("application/json");
res.setStatus(201);
res.getWriter().write(
    "{\"success\": true, \"id\": \"" + nodeName + "\"}"
);
}
}

```

8. Common Mistakes to Avoid

WRONG — Forgetting service = Servlet.class

```

// ■ WRONG
@Component
public class MyServlet extends SlingSafeMethodsServlet {}

// ■ CORRECT
@Component(service = Servlet.class)
public class MyServlet extends SlingSafeMethodsServlet {}

```

WRONG — Using SlingSafeMethodsServlet for POST

```

// ■ WRONG — doPost() won't work here
public class SaveServlet extends SlingSafeMethodsServlet {
    protected void doPost(...) { ... }
}

// ■ CORRECT
public class SaveServlet extends SlingAllMethodsServlet {
    protected void doPost(...) { ... }
}

```

WRONG — Not setting content type

```
// ■ WRONG
response.getWriter().write("{\"key\":\"value\"}");

// ■ CORRECT
response.setContentType("application/json");
response.setCharacterEncoding("UTF-8");
response.getWriter().write("{\"key\":\"value\"}");
```

9. Quick Decision Guide

Scenario	Use This
Handle a specific component/resource type?	@SlingServletResourceTypes ■
Standalone API endpoint (/bin/...)?	@SlingServletPaths
Read-only (GET/HEAD)?	extend SlingSafeMethodsServlet
Need POST/PUT/DELETE?	extend SlingAllMethodsServlet
Path-based servlet in production?	Whitelist in Dispatcher + add auth